

23.在采用页式虚拟存储管理方式的系统中，当发生上下文切换时，下列寄存器中操作系统不需要更新的是()。

- A.页表基址寄存器
- B.通用寄存器
- C.程序计数器
- D.中断向量表的基址寄存器

【答案】D 内核中断向量表基址寄存器

【分析】进程上下文切换的概念：进程上下文切换是指从一个正在运行的进程切换到另一个进程运行。在这个过程中，需保存当前进程的状态,并恢复下一个进程的状态。

A.页表基址寄存器：不同进程有不同的页表，在进程切换时，页表基址寄存器需要更新为新进程的页表基址，以便正确地进行虚拟地址到物理地址的转换。

B.通用寄存器：通用寄存器保存了当前进程的临时数据。在进程切换时，需要保存当前进程的通用寄存器内容，并恢复下一个进程的通用寄存器内容，以保证进程的正确执行。

C.程序计数器：程序计数器保存了当前进程下一条要执行的指令地址。在进程切换时，需要保存当前进程的程序计数器内容，并恢复下一个进程的程序计数器内容，以保证下一个进程从正确的位置开始执行。

D.中断向量表的基址寄存器：中断向量表是系统级的数据结构,用于处理中断。它与具体的进程无关，在进程上下文切换时不需要改变。

24.关于虚拟化技术，错误的说法是()。

- A.操作系统可以在虚拟机上运行
- B.可以用一台主机上模拟多种 ISA
- C.VMM 与操作系统特权级相同
- D.一台主机可以支持多个虚拟机

【答案】C. VMM 与操作系统特权级相同

【分析】虚拟化概念:虚拟化技术是一种将物理资源(如计算机硬件)转化为多个虚拟资源的技术，使得多个操作系统或应用程序可以在同一物理硬件上独立运行，就好像它们在独立的物理机器上运行一样。

A.操作系统可以在虚拟机上运行：正确。虚拟化技术允许在虚拟机中安装和运行操作系统，通过虚拟机监控器(VMM)来管理和分配硬件资源，使操作系统以为自己独占硬件资源。

B.可以用一台主机上模拟多种 ISA(指令集架构)：正确。通过虚拟化技术，在一台主机上可以模拟不同的 ISA，例如可以在 x86 主机上模拟 ARM 架构的虚拟机，从而运行 ARM 架构的软件。

C.VMM(虚拟机监控器)与操作系统特权级相同：**错误**。VMM 的特权级高于操作系统。VMM 负责管

理和分配硬件资源给虚拟机和其中的操作系统，它需要对硬件有完全的控制能力，包括拦截虚拟机中操作系统对硬件资源的访问请求并进行适当的处理，所以 VMM 的特权级高于操作系统。

D.一台主机可以支持多个虚拟机：正确。这是虚拟化技术的一个重要应用场景，通过合理分配主机的硬件资源(如 CPU 内存、存储等)，一台主机可以同时支持多个虚拟机运行不同的操作系统和应用程。

25.根据优先级调度算法，用队列保存就绪队列，高优先级进程在队头、队列的长度为 n ，则插入、出队进程的时间复杂度分别为()。

- A. $O(1)$, $O(1)$
- B. $O(1)$, $O(n)$
- C. $O(n)$, $O(1)$
- D. $O(n)$, $O(n)$

【答案】C. $O(n)$, $O(1)$

【分析】

插入操作：在基于优先级调度算法且用 ready 队列保存就绪态进程，高优先级进程在队头，低优先级进程在队列尾部。插入 1 个进程时，需要根据其优先级确定其在队列中位置。为了保证高优先级进程能优先被处理，插入时需要对头或队尾开始遍历队列，比较新插入进程与已有进程的优先级，直到找到合适的位置插入，这个过程最坏情况下需要遍历整个队列，也就是要遍历队列中 n 个元素，所以插入进程的时间复杂度是 $O(n)$ 。

出队操作：出队操作就是移除具有最高优先级的队头元素。对于队列这种数据结构，不管队列长度是多少，只需要进行固定的几步操作（如调整队头指针等）就能完成出队操作，故时间复杂度 $O(1)$ 。

26.采用 LRU 调度算法，局部置换策略，给进程分配 3 个页框，页面访问序列为{0, 1, 2, 0, 5, 1, 4, 3, 0, 2, 3, 2, 0}，其中 0, 1, 2 已调入内存，问缺页次数为()。

- A.5
- B.6
- C.7
- D.8

【答案】B. 6

27.确定进程运行所需的最少页框数时，要考虑的指标是()。

- A.代码段长
- B.虚拟地址空间大小
- C.物理地址空间大小
- D.指令系统支持的寻址方式

【答案】D.指令系统支持的寻址方式

【分析】

A.代码段长:代码段长只是进程代码部分的长度。进程运行需要考虑的不仅仅是代码,还包括数据段、栈段等。只看代码段长不能完整地确定进程运行所需的最少页框数。例如,一个进程可能代码段很短,但数据量很大,需要更多的页框来存储数据。

B.虚拟地址空间大小:虚拟地址空间大小是一个重要因素,但不是唯一决定因素。虽然虚拟地址空间大可能意味着需要更多的页框,但这只是一种可能性。还需要考虑进程的实际内存访问模式等因素。

C.物理地址空间大小:物理地址空间大小限制了系统总的物理内存资源,即页框的总数。但它不能直接决定一个进程运行所需的最少页框数。它只是提供了一个上限约束。

D.指令系统支持的寻址方式:对确定进程运行所需的最少页框数有重要作用,指令结构(单操作数、多操作数、寻址方式(直接寻址、间接寻址)决定了进程能够访问的内存范围和方式。参见“Operating System Concepts 9.5 讲义 PPT 示例”。

28.关于虚拟文件系统,正确的是()。

- A.虚拟文件系统是运行在虚拟内存的文件系统
- B.VFS 可以加快文件系统的访问速度
- C.VFS 定义了可访问不同文件系统的统一接口
- D.VFS 只能访问本地文件系统,不能访问网络文件系统

【答案】C.VFS 定义了可访问不同文件系统的统一接口

【分析】

VFS 概念:虚拟文件系统是一种抽象层,它位于操作系统内核和具体的文件系统(如 ext4、NTFS 等)之间。它的主要目的是为不同的文件系统提供一个统一的接口,使得操作系统能够以一种统一的方式处理各种不同类型的文件系统。

A.虚拟文件系统是运行在虚拟内存的文件系统:VFS 并不是运行在虚拟内存中的文件系统,是一个软件层,用于屏蔽不同文件系统的差异,和虚拟内存没有直接关系。

B.VFS 可以加快文件系统的访问速度:VFS 主要功能不是加快文件系统的访问速度,它的重点是提供统一的接口,虽然在某些情况下可能会因为其良好的架构间接影响性能,但这不是它的主要设计目的。

C.VFS 定义了可访问不同文件系统的统一接口:正确。VFS 提供了一组标准的函数接口,如打开文件、读取文件、写入文件等操作。这些接口对于不同的实际文件系统(如 FAT、NTFS、ext4 等)都有对应的实现。这样,操作系统的上层软件(如应用程序)在访问文件时,不需要关心底层文件系统的具体细节,只需要调用 VFS 提供的统一接口即可。

D.VFS 只能访问本地文件系统,不能访问网络文件系统:错误。VFS 不仅可以访问本地文件系统,也可以通过网络文件系统(NFS、CIFS 等)的相关模块和协议实现,访问网络上的文件系统。它提供的统一接口可以扩展到对网络文件系统的访问。

29.某文件系统采用索引节点方式。用户在目录中新建文件 F 时，文件系统不会做的是()。

- A.初始化文件 F 的索引节点
- B.在目录文件中写入 F 的索引节点号
- C.在目录文件中写入 F 的访问权限信息
- D.在目录文件中增加一条文件 F 对应的目录项

【答案】C.在目录文件中写入 F 的访问权限信息

【分析】采用索引节点 inode 的文件系统：索引节点用于存储文件的元数据，如文件大小、文件的物理块位置、访问权限等信息。目录文件主要包含文件名和对应的索引节点号，用于通过文件名查找文件的索引节点。

A.初始化文件 F 的索引节点：当新建文件 F 时，文件系统需要初始化文件 F 的索引节点，包括分配索引节点号、设置一些初始的元数据，如文件大小初始为 0、设置文件的创建时间等。

B.在目录文件中写入 F 的索引节点号：文件系统会在目录文件中为新建的文件 F 写入对应的索引节点号。这样，当通过文件名查找文件时，可以通过目录文件中的索引节点号找到文件的索引节点，进而获取文件的详细信息。

C.在目录文件中写入 F 的访问权限信息：访问权限信息是存储在文件的索引节点中的，而不是直接写入目录文件。目录文件主要是记录文件名和索引节点号的对应关系，用于查找文件的索引节点。当需要获取访问权限信息时，通过索引节点来查看的。

D.在目录文件中增加一条文件 F 对应的目录项：文件系统会在目录文件中增加一条文件 F 对应的目录项，这个目录项主要包含文件名和对应的索引节点号，用于后续的文件查找。

30.关于内存映射文件，正确的是()。

- I.可实现进程间通信
- II.实现了页面到磁盘块的映射
- III.将文件映射到进程的虚拟地址空间
- IV.将文件映射到系统的物理地址空间

【答案】D. I,II, III

【分析】

I.可实现进程间通信：内存映射文件可用于进程间通信。多个进程可以将同一个文件映射到各自的虚拟地址空间。当一个进程对映射区域进行写操作时，其他进程如果也映射了该文件，就能读取到更新后的数据，实现了数据共享和通信。

例如，在数据库系统中，多个进程可能需要同时访问和更新数据库文件。通过内存映射文件，这些进程可以共享对数据库文件的访问，一个进程对数据的修改能及时被其他进程看到，这比传统的进程间通信方式更高效，因为减少了数据复制等操作。

II.实现了页面到磁盘块的映射：内存映射文件在一定程度上涉及页面到磁盘块的映射。当进程访问内存映射文件中的虚拟地址对应的页面时，如果该页面不在物理内存中（发生缺页中断），操作系统会根据文件在磁盘上的存储结构（文件系统中磁盘块的分配情况），将对应的磁盘块内容加载到物理内存页面中。例如，在操作系统中，文件是以磁盘块为单位存储在磁盘上的。当内存映射文件被访问时，虚拟内存系统会根据文件的索引信息（如 `inode` 中的磁盘块指针）确定需要加载哪个磁盘块到物理内存页面，以满足进程对文件内容的访问需求。

III.将文件映射到进程的虚拟地址空间：内存映射文件的基本操作。通过 `mmap` 系统调，文件的全部或部分内容被映射到进程的虚拟地址空间的特定区域。例如，应用程序可以指定一个文件和虚拟地址空间中的一个范围，然后通过 `mmap` 调用将文件内容映射到该范围。此后，进程可以像访问物理内存一样访问这个虚拟地址区域来操作文件内容，操作系统会自动处理虚拟地址到物理地址的转换以及文件内容的加载和更新。

IV.将文件映射到系统的物理地址空间：错误。文件首先是映射到进程的虚拟地址空间。当进程访问被映射的虚拟地址时，通过虚拟内存管理机制(如页表)，将虚拟页面转换为物理页面，这个过程是动态的，不是直接将文件映射到物理地址空间。例如，在没有进程访问被映射文件的某个部分时，这部分内容不需要加载到物理内存，只有在进程实际访问相应虚拟地址时，才会根据需要将文件内容从磁盘加载到物理内存对应的物理地址空间部分。

31.可用于记外存空间使用情况的是()。

- A.目录
- B.系统打开文件表
- C.文件分配表 FAT
- D.进程控制块

【答案】C.文件分配表 FAT

【分析】

A.目录：目录主要用于记录文件的名称、类型、大小等基本信息，以及文件在文件系统中的层次结构。它的主要作用是方便用户和系统查找文件，而不是记录外存空间的使用情况。例如，在一个文件夹(目录)中，可以看到其中包含的文件和子文件夹的名称，但无法直接得知外存空间的具体占用情况。

B.系统打开文件表：主要用于记录系统中当前打开的文件的相关信息，如文件的打开模式(读、写等)、文件指针位置等。是为了管理正在被进程使用的文件，与外存空间的整体使用情况无直接关系。

C.文件分配表 FAT：一种用于记录文件在外存（如磁盘）上存储位置的数据结构。它通过记录文件占用的磁盘块（簇）的链接情况，不仅能够帮助系统找到文件存储的具体位置，还能够直观地反映外存空间哪些部分被文件占用，哪些部分是空闲的。通过查看 FAT 表，可以知道每个磁盘块的使用状态，从而统计外存空间的使用情况。

D.进程控制块：PCB 主要用于记录进程的各种信息，如进程状态、程序计数器、寄存器内容等。它是用于操作系统对进程进行管理的工具，与外存空间的使用情况无直接关系。

32.文件系统要为温彻斯特硬盘、固态硬盘都有的功能是()。

- A.划分扇区
- B.确定盘块大小
- C.降低寻道时间
- D.实现均衡磨损

【答案】B.确定盘块大小

A.划分扇区：温彻斯特硬盘（传统机械硬盘）通常有物理扇区的划分，这是由硬盘的物理结构决定的。而固态硬盘的存储单元和访问方式与机械硬盘不同，虽然在逻辑层面也可能有类似扇区的概念，但不是像机械硬盘那样基于物理磁道和扇区的划分。固态硬盘的存储是基于闪存芯片的闪存单元，它的基本读写单位是页(Page)和块(Block)，和机械硬盘的扇区划分有本质区别。

B.确定盘块大小：对温彻斯特硬盘、固态硬盘，文件系统都需要确定盘块大小。在文件系统中，盘块(也叫簇 cluster，不同文件系统叫法可能不同)是分配存储空间的基本单位。文件系统通过确定盘块大小来管理文件的存储和分配。对于温彻斯特硬盘，盘块大小影响文件存储的空间利用率和读写性能；对于固态硬盘，同样需要确定合适的盘块大小来管理闪存空间，以平衡读写性能、空间利用率和闪存寿命等因素。

C.降低寻道时间：寻道时间是温彻斯特硬盘特有的概念。寻道时间是指磁头从一个磁道移动到另一个磁道的时间，这是机械硬盘由于其物理结构（磁头、磁道等）产生的性能指标。

固态硬盘没有磁头和磁道，不存在寻道时间的概念，它的数据读取是通过电信号直接访问闪存芯片中的存储单元，所以降低寻道时间不是固态硬盘和温彻斯特硬盘都有的功能。

D.实现均衡磨损：均衡磨损是固态硬盘特有的需求。固态硬盘使用闪存芯片存储数据，闪存芯片有写入寿命限制(P/E 次数)。为了避免某些存储单元过度写入而提前损坏，固态硬盘需要通过技术手段(如磨损均衡算法)来均衡各个存储单元的写入次数。

45. (9 分) 三个人一起植树，甲使用铁锹挖坑，乙放树苗入坑并使用铁锹填土，丙负责为新种树苗浇水。步骤依次为：挖树坑，放树苗，填土和浇水。现在有铁锹和水桶各一个，铁锹用于挖树坑、填土。水桶用于浇水。当空树坑数量小于 3 时，甲才可以挖树坑。设初始时，树坑数目为 0，铁锹、水桶均可用。定义尽可能少的信号量，用 wait()和 signal()操作描述植树过程中三人的同步互斥关系，并说明所用信号量的作用及其初值。

【答案】

注意：要求空树坑数量小于 3，才可以挖树坑，即最多只允许有 2 个空坑，。
信号量设置：

- mutex：控制甲、乙互斥使用铁锹二元信号量，也用于同步甲乙间的动作。
初值为 1，表示初始时铁锹可用。
- empty：多元信号量，表示允许甲挖的空树坑的数量，初值 2。
- full：多元信号量，表示已有空树坑的数量，初值 0。
- syncB-C：二元信号量，代表当前是否有树苗需要浇水，初值 0。用于乙-丙之间同步，乙将树苗埋入坑后，允许丙来浇水。

甲	乙	丙
<pre>wait(empty); //判断是否允许挖坑 wait(mutex); //获取铁锹 挖坑; signal(mutex); signal(full);</pre>	<pre>wait(full); //判断有无空树坑 wait(mutex); //获取铁锹 放树苗，填坑; signal(mutex); signal(empty); signal(syncB-C);</pre>	<pre>wait(syncB-C); 浇水;</pre>

46. (6分) 某进程的虚拟地址空间如图，阴影部分为未占用区域，有 C 程序：

<pre>1 char *ptr; 2 void main() { 3 int length; 4 ptr = (char*)malloc(100); 5 scanf("%s", ptr); 6 length=strlen(ptr); 7 printf("length %d\n", length); 8 free(ptr); 9 }</pre>	<table><tr><td>操作系统内核区</td></tr><tr><td>用户栈</td></tr><tr><td> </td></tr><tr><td>共享库的存储映射区</td></tr><tr><td> </td></tr><tr><td>动态生成的堆（运行时由 malloc 创建）</td></tr><tr><td>读/写数据段</td></tr><tr><td>只读代码段</td></tr><tr><td>未使用区</td></tr></table>	操作系统内核区	用户栈		共享库的存储映射区		动态生成的堆（运行时由 malloc 创建）	读/写数据段	只读代码段	未使用区
操作系统内核区										
用户栈										
共享库的存储映射区										
动态生成的堆（运行时由 malloc 创建）										
读/写数据段										
只读代码段										
未使用区										

(1)上述程序执行时，PCB 位于哪个区域？

执行 scanf()等待键盘输入时，该进程处于什么状态？

(2)main()函数的代码位于哪个区域？

其直接调用的哪些函数的功能需要通过执行驱动程序实现？

(3)变量 ptr 被分配在哪个区域？

若变量 length 没有被分配在寄存器中，则会被分配在哪个区域？

ptr 指向的字符串位于哪个区域？

【参考答案】

(1)（2分）PCB 位于内核区，执行 scanf()时，进程处于 waiting 阻塞态。

程序执行 scanf，等待外部键盘输入，此时进程处于阻塞状态，直到输入完成。

(2)（2分）main()函数的代码位于只读代码段。

调用的 scanf()和 printf()需要执行设备驱动程序 driver，因为这些标准输入输出函数涉及到与键盘、显示器等 I/O 设备的交互，需要执行驱动程序完成实际输入输出操作。

(3)（2分）ptr 被分配在可读写数据段，length 在用户栈，ptr 指向的字符串在运行时堆。

ptr 是通过 malloc 动态分配内存得到，被分配在可读写数据段。

length 是在 main 函数内部定义的局部变量，被分配在用户栈上。

ptr 通过 malloc 在堆上分配内存得到的，指向的字符串在运行时堆上。